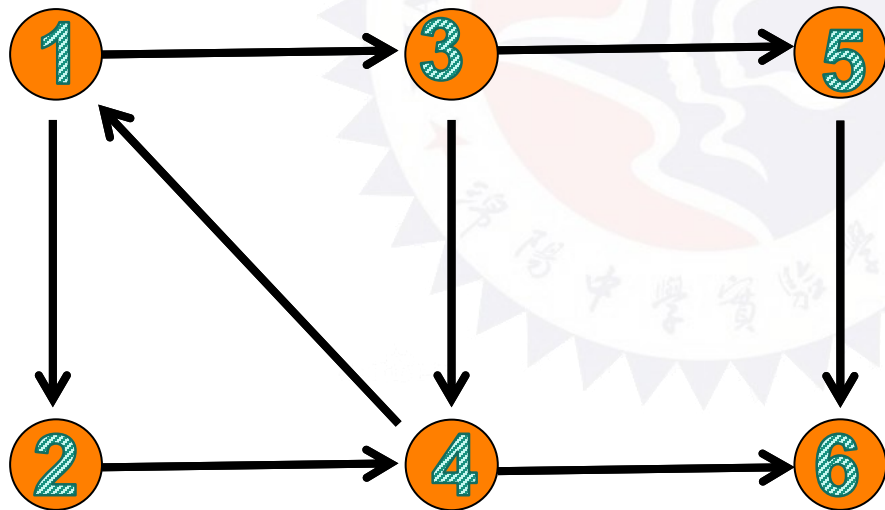


强连通分量和缩点

→ 强连通分量

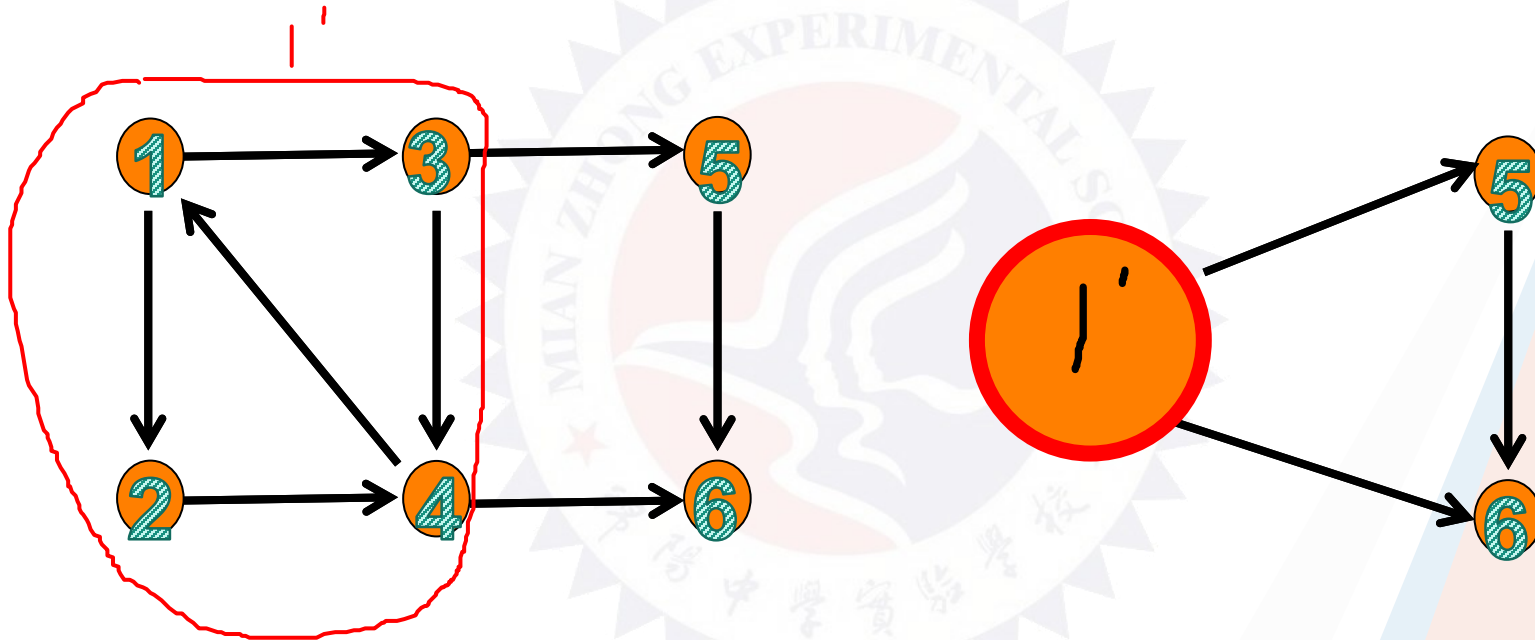
在有向图中，若存在一条 u 到 v 的路径且同时存在一条 v 到 u 的路，则称 u 和 v 是连通的。如果图中任意两点都是连通，则称该图强连通图。

有向非强连通图的**极大**强连通子图，称为强连通分量。



→ 强连通分量

若将有向图中的强连通分量都缩成一个点，则原图会形成一个**有向无环图**。



<https://www.cnblogs.com/reddest/p/5932153.html>

→ 强连通分量

求强连通分量方法:

Kosaraju和tarjan

第一种: kosaraju, 基于两次dfs, 时间复杂度为 $O(n+m)$

第二中: 一次dfs, 速度比第一种快。

学习链接: <https://www.cnblogs.com/reddest/p/5932153.html>

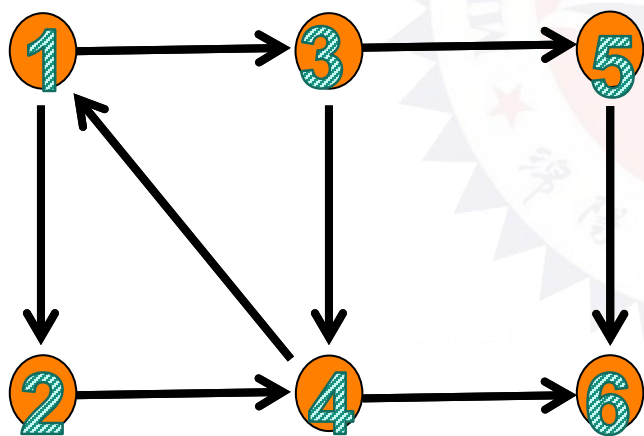
→ 强连通分量

Kosaraju: 对于每一个强连通, u 和 v 能相互到达。我们将图的所有边反向后, u 依然能走到 v 。

步骤: 1、从任意点开始dfs, 将访问完的点加入栈sta。

2、从栈顶开始在反向图中dfs, 删除每个访问的点, 这些访问到的点构成一个强连通分量。

3、若栈不为空, 继续第二步。



搜索顺序: 1, 2, 4, 6, 3, 5;

回溯顺序: 6, 4, 2, 5, 3, 1;

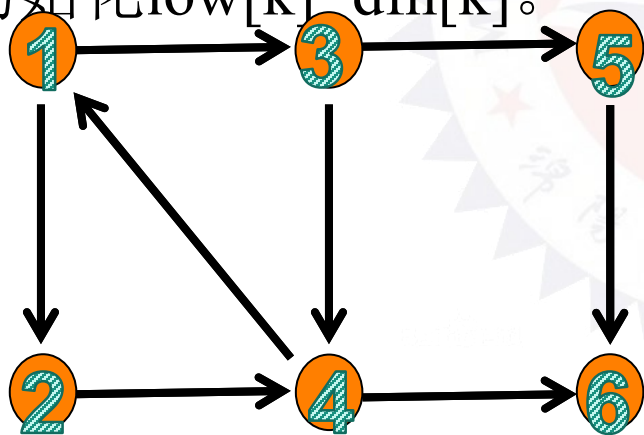
→ 强连通分量

tarjan: 基于1次dfs的算法，搜索时将搜索的点入栈。为每个强连通找一个“根”，回溯时，若发现k点是根，则k点所在的强连通全部出栈。所以k点必须是这个强连通所有成员里最先入栈的点。即搜索树里的祖先。

dfn[k] 存储k的搜索顺序编号（时间戳）

low[k] 存储k或者k的子树能搜索到的最早的点的dfn值。即最早的祖先。

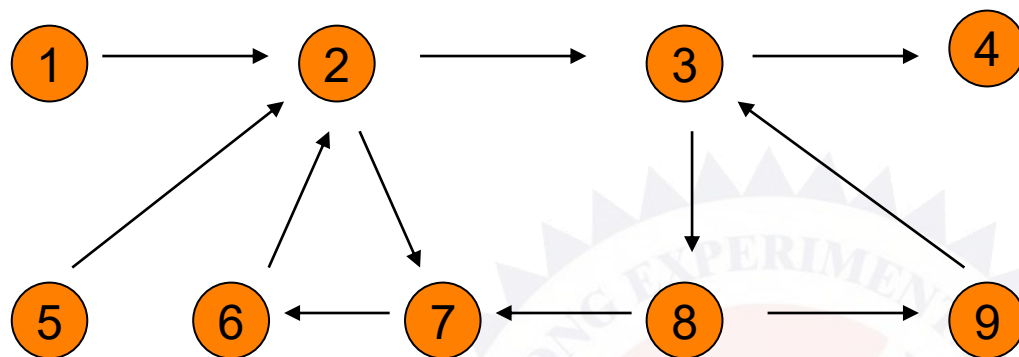
初始化 $low[k] = dfn[k]$ 。



搜索顺序: 1, 2, 4, 6, 3, 5;

当搜索到4的时候，有一条指向祖先的边，则 $low[4] = dfn[1]$; 回溯到2的时候， $low[2] = low[4]$; 在这个强连通里，只有1点的low等于dfn的。

→ 强连通分量



假如给的边满足搜索顺序：
1,2,7,6,3,8,9,4;

1



→ 缩点

每头奶牛都梦想成为牛棚里的明星。被所有奶牛喜欢的奶牛就是一头明星奶牛。所有奶牛都是自恋狂，每头奶牛总是喜欢自己的。奶牛之间的“喜欢”是可以传递的——如果A喜欢B，B喜欢C，那么A也喜欢C。牛栏里共有N头奶牛，给定一些奶牛之间的爱慕关系，请你算出有多少头奶牛可以当明星。

输入格式

第一行：两个用空格分开的整数：N和M

第二行到第M + 1行：每行两个用空格分开的整数：A和B，表示A喜欢B

输出格式

第一行：单独一个整数，表示明星奶牛的数量

【数据范围】 100%的数据 $N \leq 10000, M \leq 50000$



→ 缩点

每头奶牛都梦想成为牛棚里的明星。被所有奶牛喜欢的奶牛就是一头明星奶牛。所有奶牛都是自恋狂，每头奶牛总是喜欢自己的。奶牛之间的“喜欢”是可以传递的——如果A喜欢B，B喜欢C，那么A也喜欢C。牛栏里共有N头奶牛，给定一些奶牛之间的爱慕关系，请你算出有多少头奶牛可以当明星。

输入格式

第一行：两个用空格分开的整数：N和M

第二行到第M + 1行：每行两个用空格分开的整数：A和B，表示A喜欢B

输出格式

第一行：单独一个整数，表示明星奶牛的数量

【数据范围】 100%的数据 $N \leq 10000, M \leq 50000$



→ 练习题目网络协议

- 描述一些学校的校园网连接在一个计算机网络上。学校之间存在软件支援协议。每个学校都有它应支援的学校名单（学校a支援学校b，并不表示学校b一定支援学校a）。当某校获得一个新软件时，无论是直接得到的还是从网络得到的，该校都应立即将这个软件通过网络传送给它应支援的学校。因此，若需要让所有连接在网络上的学校都能使用一个新软件，只需要将其提供给其中一些学校即可
- 子任务a：根据学校间软件支援协议（各个学校的支援名单），计算最少需要将一个软件直接提供给多少个学校，才能使该软件通过网络传送到所有学校。
- 子任务b：如果允许在原有支援协议上添加新的支援关系，则总可以形成一个新的协议，使得此时只需要将一个新软件提供给任何一个学校，其他所有学校就都可以通过网络获得该软件。请计算出最少需要添加几条新的支援关系。

练习题目网络协议

输入格式

- 第一行是一个整数 $n(2 \leq n \leq 100)$ ，表示与网络连接的学校总数。接下来 n 行描述了每个学校要支援的学校。第 $i+1$ 行表示第 i 号学校要支援的所有学校的编号，编号之间用空格隔开，每行以数字 0 结束。如果某个学校不支援任何学校，则相应的行会有一个 0。

输出格式

- 包含两行，第一行是一个正整数，表示子任务 a 的解。第二行也是一个正整数，表示子任务 b 的解。

样例输入1

1	5
2	2 4 3 0
3	4 5 0
4	0
5	0
6	1 0

样例输出1

1	1
2	2

→ 练习题目—消息的传递

- 我们的郭嘉大大在曹操这过得逍遥自在，但是有一天曹操给了他一个任务，在建邺城内有 N 个袁绍的奸细，将他们从 1 到 N 进行编号，同时他们之间存在一种传递关系，即若 $c[i][j]=1$ ，则奸细 i 能将消息直接传递给奸细 j 。
- 现在曹操要发布一个假消息，需要传达给所有奸细，而我们的郭嘉大大则需要传递给尽量少的奸细使所有的奸细都知道这个消息，问我们至少要传给几个奸细？
- **输入格式**
- **文件的第一行为 N ，第二行至第 $N+1$ 行为 的矩阵（若第 i 行第 j 列为 1，则奸细 i 能将消息直接传递给奸细 j ，若第 i 行第 j 列为 0，则奸细 i 不能将消息直接传递给奸细 j ）。**
- **输出格式**
- **输出文件只有一行：即我们的郭嘉大大首先至少要传递的奸细个数**



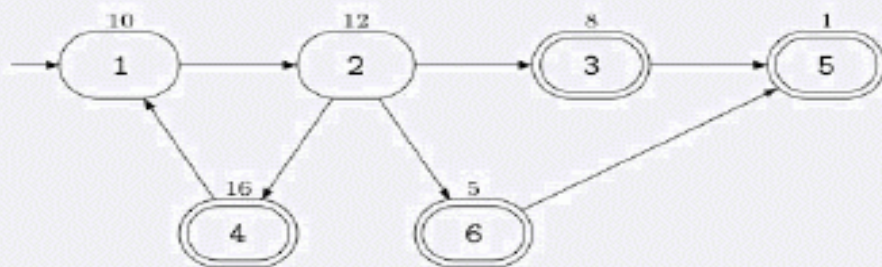
练习题目—搬运计划

Siruseri 城中的道路都是单向的。不同的道路由路口连接。按照法律的规定，在每个路口都设立了一个 Siruseri 银行的 ATM 取款机。令人奇怪的是，Siruseri 的酒吧也都设在路口，虽然并不是每个路口都设有酒吧。

Banditji 计划实施 Siruseri 有史以来最惊天动地的 ATM 抢劫。他将从市中心出发，沿着单向道路行驶，抢劫所有他途径的 ATM 机，最终他将在一个酒吧庆祝他的胜利。

使用高超的黑客技术，他获知了每个 ATM 机中可以掠取的现金数额。他希望你帮助他计算从市中心出发最后到达某个酒吧时最多能抢劫的现金总数。他可以经过同一路口或道路任意多次。但只要他抢劫过某个 ATM 机后，该 ATM 机里面就不会再有钱了。

例如，假设该城中有 6 个路口，道路的连接情况如下图所示：



市中心在路口 1，由一个入口符号→来标识，那些有酒吧的路口用双圈来表示。每个 ATM 机中可取的钱数标在了路口的上方。在这个例子中，Banditji 能抢劫的现金总数为 47，实施的抢劫路线是：1-2-4-1-2-3-5。



→ 练习题目—搬运计划

- **输入格式**
- 第一行包含两个整数 N 、 M 。 N 表示路口的个数， M 表示道路条数。接下来 M 行，每行两个整数，这两个整数都在1到 N 之间，第 $i+1$ 行的两个整数表示第 i 条道路的起点和终点的路口编号。接下来 N 行，每行一个整数，按顺序表示每个路口处的ATM机中的钱数。接下来一行包含两个整数 S 、 P ， S 表示市中心的编号，也就是出发的路口。 P 表示酒吧数目。接下来的一行中有 P 个整数，表示 P 个有酒吧的路口的编号
- **输出格式**
- 输出一个整数，表示Banditji从市中心开始到某个酒吧结束所能抢劫的最多的现金总数。



→ 练习题目—搬运计划

- 输入

- 6 7
1 2
2 3
3 5
2 4
4 1
2 6
6 5
10
12
8
16
1 5
1 4
4
3
5
6

- 输出

- 47





→ 练习题目

- 洛谷 p1262 间谍网络
- 洛谷 p2341 受欢迎的牛



The background features a central point from which several wide, colorful rays emanate, creating a starburst effect. The rays are in shades of blue, orange, red, and grey. Faint binary code (0s and 1s) is scattered across the background, and a small line graph with two fluctuating lines is visible on the right side.

Thank you!